SOFTWARE REQUIREMENTS SPECIFICATION

SkillLens

A Skill-to-Employment Intelligence Platform for Indian Engineering Students

Version	1.0
Date	March 2026
Project	Final Year Major Project — B.E. Computer Science
Institute	Swami Vivekanand College of Engineering, Indore
Mentor	Ms. Amisha Jain
Authors	Teena Devray · Rohit Solanki · Vishal Panwar

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for SkillLens — a web-based career preparation platform targeting Indian engineering students preparing for campus placements. The document serves as the authoritative reference for design, development, and testing decisions.

1.2 Project Overview

SkillLens addresses a critical gap in the Indian placement preparation ecosystem. Existing tools — Naukri, LinkedIn, Resumod — either scan resumes for keyword presence or provide generic interview practice. None measure how deeply a student knows each skill, nor do they map that depth to specific companies' real requirements.

SkillLens combines five tightly integrated modules into a single feedback loop:

- Smart Resume Analyzer — extracts skills and measures depth via quiz
- Skill Depth Quiz Engine — 10 MCQs per skill, producing a depth percentage
- Company Match Engine — maps skill depth to 25 real Indian company profiles
- AI Mock Interview — evaluates technical accuracy and communication separately
- Placement Roadmap — generates a curated, resource-linked weekly plan

1.3 Scope

SkillLens is a browser-based MERN stack application. It targets final-year and pre-final-year engineering students in Tier 2 and Tier 3 Indian cities. The system is not a job portal, does not connect to live job APIs, and does not perform automated resume submission. It is a preparation and self-assessment tool.

1.4 Definitions

Term	Definition
Depth Score	A percentage (0–100) representing how well a student knows a skill, measured by quiz performance — not keyword presence.
Readiness Score	A composite score (0–100) synthesising depth scores, interview performance, roadmap completion, and company match.
Company Profile	A curated JSON object containing a company's required skills, their weights, and minimum depth thresholds — built from real job postings.
Gap Analysis	The identification of which specific skills, and by how much, are lowering a student's match score for a target company.
Resource Library	A static curated JSON of one video, one article, one practice link, and one task per skill per level.

1.5 References

- React.js — https://react.dev
- Node.js — https://nodejs.org/en/docs
- Express.js — https://expressjs.com
- MongoDB — https://www.mongodb.com/docs
- OpenAI API — https://platform.openai.com/docs
- pdf-parse NPM package — https://www.npmjs.com/package/pdf-parse
- JSON Web Tokens — https://jwt.io/introduction
- Tailwind CSS — https://tailwindcss.com

2. Overall Description

2.1 Product Perspective

SkillLens is a standalone web application with no dependency on external job platforms. It operates as a self-contained system: the student inputs data (resume, quiz answers, interview answers), the system processes and scores them, and returns structured, actionable feedback. The only external service consumed is the OpenAI API, used exclusively for interview answer evaluation.

2.2 User Classes

User Type	Description	Primary Interaction
Student	Engineering student preparing for campus placements	All five modules — primary user
College Admin	Placement cell coordinator (future scope)	Batch analytics dashboard
System Admin	Developer maintaining company profiles and resource library	JSON data files only

2.3 Operating Environment

- Frontend: React.js, Tailwind CSS, Axios, Recharts, React Router
- Backend: Node.js, Express.js, Multer (file upload), pdf-parse, JWT, bcrypt
- Database: MongoDB with Mongoose ODM
- AI Integration: OpenAI API (interview evaluation only)
- Deployment: Render (backend), Vercel (frontend)
- Browser support: Chrome, Firefox, Edge — latest two versions
- Minimum client hardware: 4 GB RAM, stable internet connection

2.4 Assumptions and Dependencies

- The OpenAI API key is valid and within usage limits; interview evaluation degrades gracefully if the API is unavailable.
- Company profiles and the resource library are static JSON files, manually curated, and updated by the development team — not generated by AI at runtime.
- Resume uploads are PDF format only, with text-extractable content (scanned image PDFs are not supported).
- The skill quiz question bank covers 10 skills: React, Node.js, MongoDB, SQL, Python, Java, DSA, REST APIs, System Design basics, and Git.

---

3. System Features and Functional Requirements

3.1 User Authentication

FR-1.1 The system shall allow users to register with name, email, password, and target job role.

FR-1.2 The system shall hash passwords using bcrypt before storage.

FR-1.3 The system shall issue a JWT on successful login, valid for 7 days.

FR-1.4 All routes except /register and /login shall require a valid JWT.

FR-1.5 Users shall be able to update their target job role from their profile page.

3.2 Resume Analyzer

FR-2.1 The system shall accept PDF resume uploads up to 5 MB via a drag-and-drop interface.

FR-2.2 The system shall extract raw text from the PDF using pdf-parse.

FR-2.3 The system shall detect the presence of skills from a predefined list of 30 skills using case-insensitive string matching.

FR-2.4 For each detected skill, the system shall prompt the user to complete the corresponding skill quiz if not already taken.

FR-2.5 The system shall display detected skills, their depth scores (if quiz taken), and which required skills are missing for the user's target role.

3.3 Skill Depth Quiz Engine

FR-3.1 The system shall maintain a question bank of 10 MCQs per skill across 10 skills (100 questions total), stored in a static JSON file.

FR-3.2 Each quiz shall present 10 questions in randomised order with 4 options each.

FR-3.3 The system shall compute a depth score as: (correct answers / 10) × 100.

FR-3.4 The system shall classify depth into three levels: Beginner (0–40%), Intermediate (41–70%), Advanced (71–100%).

FR-3.5 Depth scores shall be stored per user per skill and updated on retake.

FR-3.6 The system shall recalculate the Readiness Score and company match percentages after every quiz completion.

3.4 Company Match Engine

FR-4.1 The system shall maintain profiles for 25 Indian companies in a static JSON file, each containing required skills, per-skill weights, and minimum depth thresholds.

FR-4.2 The system shall calculate a match percentage for each company using weighted skill depth scoring.

FR-4.3 The system shall display companies ranked from highest to lowest match percentage.

FR-4.4 For a selected company, the system shall display a gap analysis: the top 3 skills dragging the score down, the magnitude of each gap, and the specific action to fix it.

FR-4.5 The system shall display a plain-language verdict: the single highest-impact skill to improve first.

3.5 AI Mock Interview

FR-5.1 The system shall load role-specific interview questions from a static JSON file (5 questions per role).

FR-5.2 The system shall present questions one at a time with a text area for the student's written answer.

FR-5.3 The system shall send each question-answer pair to the OpenAI API with a structured prompt that requests a JSON response containing: technical score (1–10), communication score (1–10), technical feedback (one sentence), communication feedback (one sentence), missed key concept, a model two-sentence answer, and an array of missed technical keywords.

FR-5.4 The system shall display each field of the AI response separately — not as a text blob.

FR-5.5 Missed keywords from FR-5.3 shall be fed back to the quiz engine to flag weak skill areas, even if the student's quiz score was adequate.

FR-5.6 Interview results shall be stored per session and used to update the Readiness Score.

3.6 Placement Roadmap

FR-6.1 The system shall generate a week-by-week roadmap based on the student's gap analysis, target company, and user-specified weeks remaining before placement season.

FR-6.2 The AI shall be used only to determine the ordering and priority of skills to address — not to generate the resource links.

FR-6.3 Each week in the roadmap shall be populated from the static resource library JSON with: one YouTube video link, one article link, one practice link, and one concrete task description.

FR-6.4 Students shall be able to mark weekly tasks as complete; completion percentage shall feed into the Readiness Score.

FR-6.5 The roadmap shall regenerate automatically when the student's target company or depth scores change significantly.

3.7 Progress Dashboard

FR-7.1 The dashboard shall display a Readiness Score (0–100) computed as a weighted average: depth score average (35%), interview average (30%), roadmap completion (20%), best company match (15%).

FR-7.2 The dashboard shall display a line chart of Readiness Score over time using Recharts.

FR-7.3 The dashboard shall display per-skill depth scores as a horizontal bar chart.

FR-7.4 The dashboard shall display the student's top 3 matched companies and their current match percentages.

FR-7.5 The dashboard shall display a next-step prompt: the single action that will most improve the Readiness Score.

4. Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	API responses for resume analysis and company matching shall complete within 3 seconds on a standard broadband connection.
NFR-2	Performance	Interview evaluation (OpenAI API call) shall display a loading state and complete within 10 seconds; if the API call exceeds 15 seconds, the system shall time out gracefully and notify the user.
NFR-3	Security	All passwords shall be hashed with bcrypt (minimum 10 salt rounds) before storage. Plaintext passwords shall never be logged or returned.
NFR-4	Security	All API endpoints except /register and /login shall validate the JWT before processing. Expired or invalid tokens shall return HTTP 401.
NFR-5	Security	Uploaded resume files shall be stored temporarily, processed, and deleted within the same request cycle. No resume files shall be persisted on the server.
NFR-6	Reliability	The system shall function with degraded but usable capability if the OpenAI API is unavailable. The interview module shall notify the user and allow answer submission without AI scoring.
NFR-7	Usability	All primary user workflows (resume upload → quiz → company match → roadmap) shall be completable without any written instructions, using UI cues alone.
NFR-8	Scalability	The MongoDB schema shall use indexes on userId for all analysis and result collections to support efficient queries as the user base grows.
NFR-9	Maintainability	Company profiles, quiz questions, and the resource library shall be maintained as separate JSON files, editable without code changes or redeployment.
NFR-10	Portability	The application shall render correctly on screen widths from 375 px (mobile) to 1920 px (desktop) using responsive Tailwind CSS classes.

5. Database Design

Four MongoDB collections, all timestamped. userId is indexed on all non-user collections.

Users

Field	Type	Notes
_id	ObjectId	Auto-generated primary key
name	String	Required
email	String	Required, unique, lowercase
password	String	bcrypt hash — never plaintext
targetRole	String	e.g., 'frontend', 'backend', 'data'
createdAt	Date	Auto-timestamp

SkillProfiles

Field	Type	Notes
userId	ObjectId	Ref: Users — indexed
skill	String	e.g., 'react', 'mongodb'
depthScore	Number	0–100, updated on quiz retake
depthLevel	String	'Beginner' / 'Intermediate' / 'Advanced'
quizAttempts	Number	Count of times quiz taken
lastAttempt	Date	Timestamp of most recent quiz

InterviewResults

Field	Type	Notes
userId	ObjectId	Ref: Users — indexed

role	String	Target role for this session
question	String	The interview question text
answer	String	Student's written answer
technicalScore	Number	1–10, from OpenAI
communicationScore	Number	1–10, from OpenAI
technicalFeedback	String	One-sentence AI feedback
communicationFeedback	String	One-sentence AI feedback
missedKeywords	[String]	Array fed back to quiz engine
date	Date	Auto-timestamp

RoadmapProgress

Field	Type	Notes
userId	ObjectId	Ref: Users — indexed
week	Number	Week number in the plan
skill	String	Skill targeted this week
level	String	'beginner' or 'intermediate'
completed	Boolean	Student marked as done
completedAt	Date	Null until marked done

6. API Endpoint Reference

Method	Endpoint	Auth	Description
POST	/api/auth/register	No	Register new user
POST	/api/auth/login	No	Login; returns JWT
GET	/api/auth/profile	Yes	Get current user profile
PUT	/api/auth/profile	Yes	Update target role
POST	/api/resume/upload	Yes	Upload PDF; extract and return detected skills
GET	/api/quiz/:skill	Yes	Fetch 10 randomised questions for a skill
POST	/api/quiz/:skill/submit	Yes	Submit answers; compute and store depth score
GET	/api/companies	Yes	Return all 25 company profiles with current match %
GET	/api/companies/:id/gap	Yes	Return gap analysis for a specific company
GET	/api/interview/questions/:role	Yes	Load 5 questions for a role
POST	/api/interview/evaluate	Yes	Submit answer; call OpenAI; store and return result
GET	/api/roadmap	Yes	Return current roadmap with resources and completion status
POST	/api/roadmap/generate	Yes	Trigger AI ordering + build roadmap from resource library
PUT	/api/roadmap/:week/complete	Yes	Mark a week as complete
GET	/api/dashboard/metrics	Yes	Return all data needed for the dashboard

7. System Constraints

- OpenAI API costs are borne by the development team during the project phase; production usage requires a billing-aware rate-limiting strategy.
- Resume text extraction relies on pdf-parse, which does not support scanned or image-only PDFs.
- Company profiles and the resource library are static — they do not update automatically from live job postings.
- The skill quiz question bank is limited to 10 skills in the initial version.
- The system does not verify whether a student has actually completed a roadmap task — completion is self-reported.

8. Future Scope

- Voice-based interview simulation using a speech-to-text API (e.g., Whisper), allowing spoken answers to be evaluated.
- Live coding assessment integrated via Judge0 API, enabling in-browser code execution and automated test-case scoring.
- College administrator dashboard with batch-level analytics, allowing placement cells to track cohort readiness across departments.
- Smart skill recommendations using collaborative filtering: students with similar profiles who improved certain skills show gains, and those recommendations surface to peers.
- Mobile application using React Native, extending access to students without consistent desktop access.

- Dynamic company profiles refreshed periodically by scraping publicly listed job requirements, reducing manual curation effort.

Revision History

Version	Date	Author	Changes
1.0	March 2026	Teena Devray, Rohit Solanki, Vishal Panwar	Initial release